ENGS 3302 PATTERN RECOGNITION · STUDENT NOTES

Week 1: Introduction to Pattern Recognition

Main source: Duda, Hart & Stork, *Pattern Classification*, 2nd ed., Chapter 1 (pp. 1–17)

What you must be able to do after this week (exam scope)

1. Explain what pattern recognition is, and give examples from everyday life.
2. Use the basic terms correctly: *class, sample, feature, feature vector, feature space, training set, decision boundary*.
3. Name the stages of a pattern recognition system and say what each stage does.
4. Read histograms and scatter plots of features, and count the errors of a simple decision rule.
5. Explain *generalization*, and why a very complex decision boundary can be a bad choice.
6. Describe the design cycle of a pattern recognition system.
7. Tell supervised, unsupervised and reinforcement learning apart, with an example of each.

Not required: the related fields, *analysis by synthesis*, and syntactic pattern recognition (Duda §1.2–1.3).

Reading (optional, these notes are complete): Duda §1.1–1.6 (pp. 1–17). Free alternative for Section 1.7: James et al., *An Introduction to Statistical Learning with Python* (ISLP), §2.1.4.

1.1 What is pattern recognition?

[Duda Ch. 1 intro and §1.1, p. 1]

Every day you recognize faces, voices, handwriting and the keys in your pocket, without effort. Duda defines **pattern recognition** as

Definition (Duda, p. 1)

"the act of taking in raw data and taking an action based on the 'category' of the pattern."

In this course we build *machines* that do this. The raw data can be an image, a sound, a signal or a table of numbers. The machine must decide which **class** (category) the input belongs to.

Application	Raw data	Classes
Face unlock on a phone	camera image	owner / not the owner
Spam filter	text of an e-mail	spam / not spam
Number-plate reading	image of a car	the letters and digits 0–9, A–Z
Voice commands	sound signal	"call", "stop", "play", …
Medical screening	X-ray image	healthy / needs a doctor's review
Machine monitoring	temperature, vibration	normal / faulty

1.2 A first example: sorting fish

[Duda §1.2, pp. 1–4]

A fish factory wants a machine that sorts fish on a conveyor belt into two classes: **sea bass** and **salmon**. A camera takes a picture of each fish. We will use this example all semester. Later we will call the two classes ω_1 (sea bass) and ω_2 (salmon).

1.2.1 Choosing a feature

A **feature** is a property of the object that we can measure, such as its length, lightness or width. The workers say that sea bass are usually longer than salmon. So we measure the length of many fish whose type we already know. These fish are the **training samples**. Figure 1 shows the result.

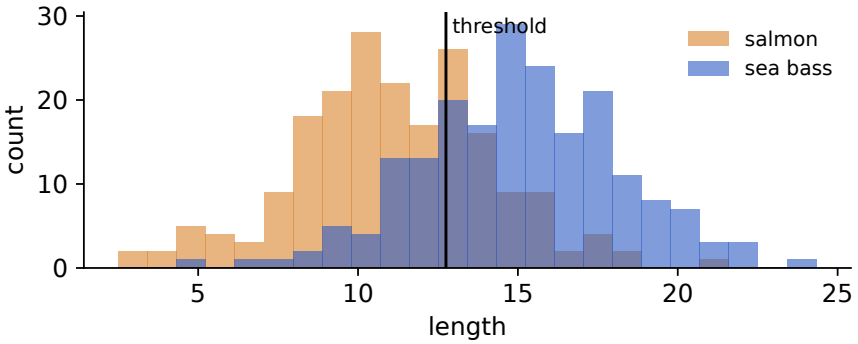


Figure 1: Length of 200 salmon and 200 sea bass. The two groups overlap a lot.

A simple rule is: *if the length is larger than a threshold, decide sea bass.* Even with the best threshold (about 12.75), this rule is wrong for about **26%** of these fish. No threshold can separate the two groups well, because they overlap too much. Length is a *poor feature* for this task.

We try another feature: the **lightness** of the scales (Figure 2). Now the groups overlap much less, and the best threshold (about 5.05) makes only about **10%** errors. Lightness is a *better feature*.

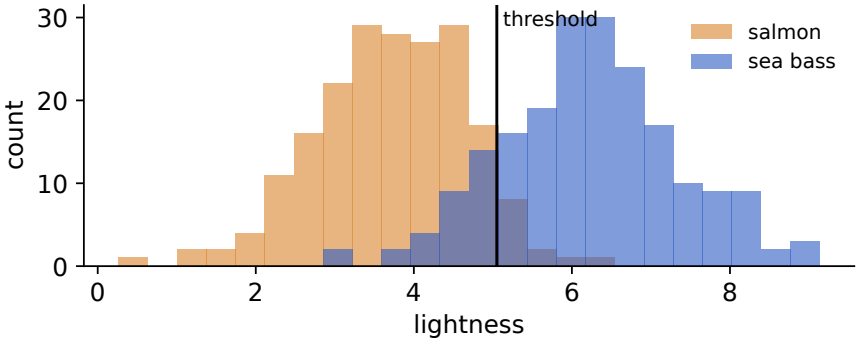


Figure 2: Lightness of the same fish. Salmon are darker; the overlap is small.

Key idea

A good feature has values that are *similar within a class* and *different between classes*. The classifier can only be as good as its features.

1.2.2 Counting errors by hand

Worked example 1.1 — ten fish, one feature

Ten training fish (lightness x_1, width x_2):

Salmon lightness	2.5	3.0	3.8	4.5	5.6
Salmon width	16.0	19.3	16.5	18.0	17.0
Sea bass lightness	4.2	5.2	6.0	6.8	7.5
Sea bass width	20.5	19.0	21.0	19.5	20.0

Rule: decide sea bass if lightness > 5.0. **Errors:** the salmon with 5.6 is called sea bass, and the sea bass with 4.2 is called salmon. That is 2 errors out of 10, an error rate of 20%.
Try the threshold 5.4: now the sea bass with 5.2 is also wrong, so 3 errors. With lightness alone we cannot do better than 2 errors here.

1.3 Feature vectors and feature space

[Duda §1.2, pp. 6–7]

Sea bass are also wider than salmon. If we measure *two* features, each fish becomes a point in a plane:

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} \text{lightness} \\ \text{width} \end{pmatrix}.$$

This column of numbers is the **feature vector**. The plane (or, with d features, the d-dimensional space) is the **feature space**. Each training fish is one point in this space.

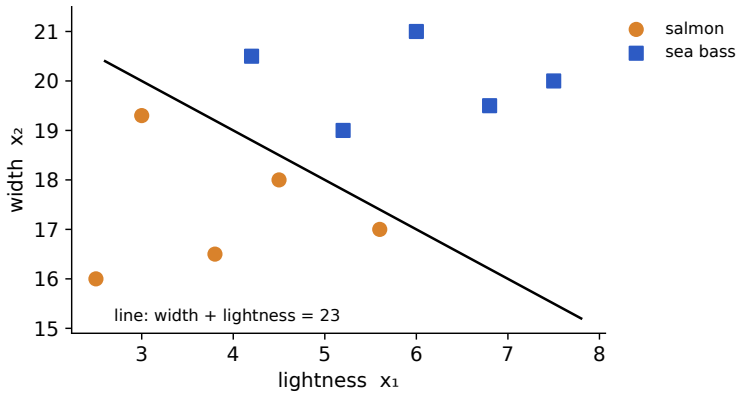


Figure 3: The ten fish of Worked example 1.1 in feature space. A straight line separates all ten.

The task of the classifier is now to *split the feature space into regions*: one region for sea bass and one for salmon. The line or curve between the regions is the **decision boundary**.

Worked example 1.2 — two features, one line

Rule: decide **sea bass** if width + lightness > 23.
Salmon (2.5, 16.0): $16.0 + 2.5 = 18.5 < 23 \Rightarrow$ salmon ✓
Sea bass (4.2, 20.5): $20.5 + 4.2 = 24.7 > 23 \Rightarrow$ sea bass ✓ (this fish was wrong with lightness alone)
Salmon (5.6, 17.0): $17.0 + 5.6 = 22.6 < 23 \Rightarrow$ salmon ✓ (also wrong before)

Checking all ten fish gives 0 errors. Width alone cannot do this either: the salmon with width 19.3 is wider than the sea bass with 19.0. Two features together did what neither feature could do alone.

In code

```
import numpy as np
X = np.array([[2.5, 16.0], [4.2, 20.5], [5.6, 17.0]])  # lightness, width
is_sea_bass = X[:, 1] + X[:, 0] > 23                     # [False, True, False]
```

Should we add more features? Duda lists more candidates: fin shape, position of the eyes, and so on. More features can help, but some are redundant (they repeat information we already have), some are expensive to measure, and too many features can even make the classifier worse. Choosing features is a central topic of this course (Weeks 2 and 9).

1.4 Decision boundaries and generalization

[Duda §1.2, pp. 7–9]

A classifier is useful only if it works on *new* fish that it has never seen. This ability is called **generalization**.

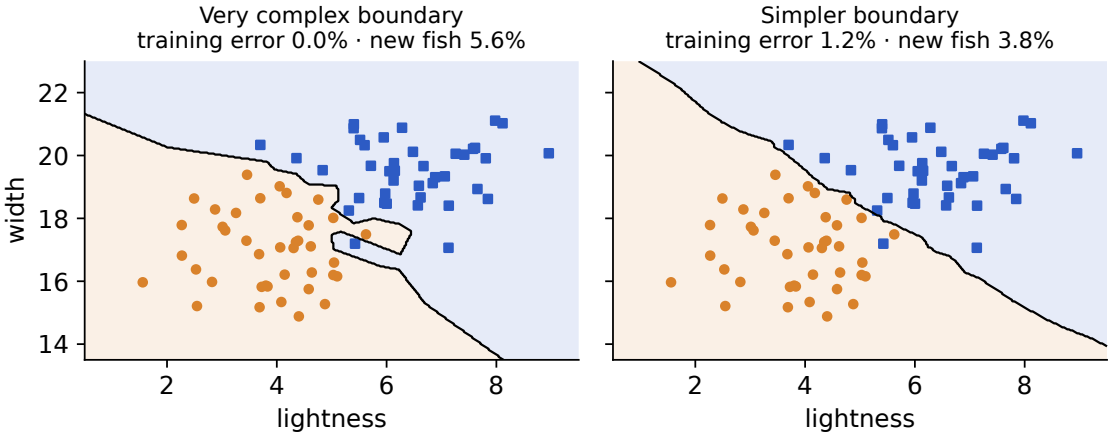


Figure 4: Left: a boundary that follows every training fish. Right: a smoother boundary. The smoother one makes a few training errors but fewer errors on 320 new fish.

In Figure 4, the left boundary bends around every training point and makes **0%** training errors, but **5.6%** errors on new fish. The right boundary is simpler. It makes **1.2%** training errors but only **3.8%** on new fish. The complex boundary learned the *accidents* of this particular training set, not the real difference between the classes.

Be careful

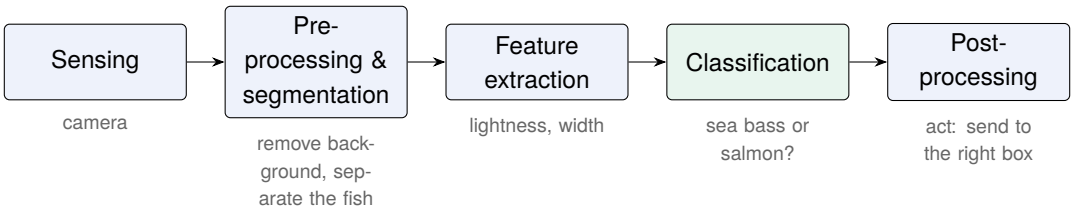
Zero errors on the training data does *not* mean a good classifier. Always judge a classifier on data it has *not* seen. We will learn how to do this properly in Week 6.

Duda connects this to an old principle, **Occam's razor**: prefer the simpler explanation unless the complex one is clearly needed.

1.5 The stages of a pattern recognition system

[Duda §1.3, pp. 9–14]

The fish sorter, like most pattern recognition systems, is a chain of stages (Duda, Figure 1.1):



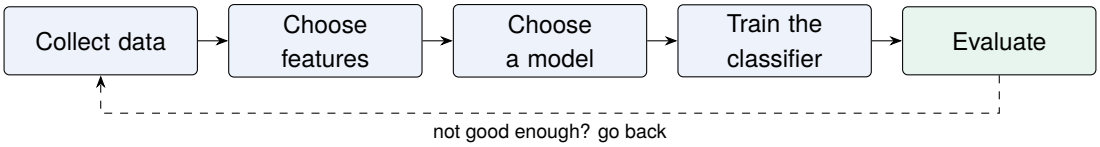
Stage	What it does
Sensing	A sensor (camera, microphone, thermometer) turns the real object into numbers. Its quality limits everything after it.
Pre-processing and segmentation	Cleans the data (for example, corrects the light level) and isolates each object from the background and from other objects.
Feature extraction	Reduces the data to a few numbers that are useful for the decision. A photo of thousands of pixels becomes, for example, two numbers.
Classification	Uses the feature vector to assign a class. This is the main subject of the course.
Post-processing	Turns the class into an action, and can use costs and context (for example, which mistakes are more expensive).

Example from another field: a number-plate reader *senses* with a camera, *segments* the plate and each character, *extracts* features of each character, *classifies* it as a letter or digit, and *post-processes* by checking that the result has a valid plate format.

1.6 The design cycle

[Duda §1.4, pp. 14–16]

How do we *build* such a system? Duda describes a cycle of steps. If the result is not good enough, we go back and change an earlier step.



Step	Key question	In this course
Collect data	Do we have enough *labelled* examples that represent real use?	Lab 1
Choose features	Which measurements separate the classes?	Weeks 2, 9
Choose a model	Which kind of classifier and boundary fits the problem?	Weeks 4–12
Train	How do we use the data to set the classifier's parameters?	Weeks 5, 7–12
Evaluate	How well will it work on new data? Is it fast and cheap enough?	Week 6

The final project follows the same idea: you will define a problem, choose features and an algorithm, build the system, and test it.

1.7 Learning: supervised, unsupervised, reinforcement

[Duda §1.5, p. 16]

Learning means using training samples to design the classifier. It comes in three forms:

Type	What the training data gives	Example
Supervised	Each sample comes with its correct class (a *label*), given by a "teacher".	Fish with known species; e-mails marked spam / not spam.
Unsupervised	No labels. The system finds natural groups (*clusters*) by itself.	Grouping customers by buying habits (Week 11).
Reinforcement	No correct answer, only a signal of *right* or *wrong* (a critic).	A game-playing program that learns from winning and losing.

Most of this course is about supervised learning. Unsupervised learning comes in Week 11. Reinforcement learning is not part of this course.

Summary

Pattern recognition	Taking raw data and acting on the category (class) of the pattern.
Feature / feature vector	A measured property / the list of features of one object, a point in feature space.
Decision boundary	The line or surface that separates the regions of the classes in feature space.
Generalization	Working well on new data. A very complex boundary often generalizes badly.
System stages	Sensing → pre-processing & segmentation → features → classification → post-processing.
Design cycle	Data → features → model → training → evaluation, and back again.
Learning	Supervised (labels), unsupervised (no labels, clusters), reinforcement (right/wrong signal).

Check yourself

1. Give one example of pattern recognition that you used today. What are the raw data and the classes?
2. In Worked example 1.1, how many errors does the rule "sea bass if lightness > 4.0" make?
3. Use the rule of Worked example 1.2 on a new fish with lightness 5.0 and width 18.5. Which class?
4. A classifier has 0% errors on its training data and 30% on new data. What went wrong?
5. For a number-plate reader, what happens in the segmentation stage?
6. Grouping songs by similarity without any labels: which type of learning is this?

Answers. (1) Example: face unlock; raw data = camera image; classes = owner / not owner. (2) Two errors: the salmon with 4.5 and the salmon with 5.6 are called sea bass. (3) $18.5 + 5.0 = 23.5 > 23 \Rightarrow$ sea bass. (4) The boundary is too complex; it does not generalize (it learned the training accidents). (5) The plate is found in the image and each character is cut out separately. (6) Unsupervised learning (clustering).